

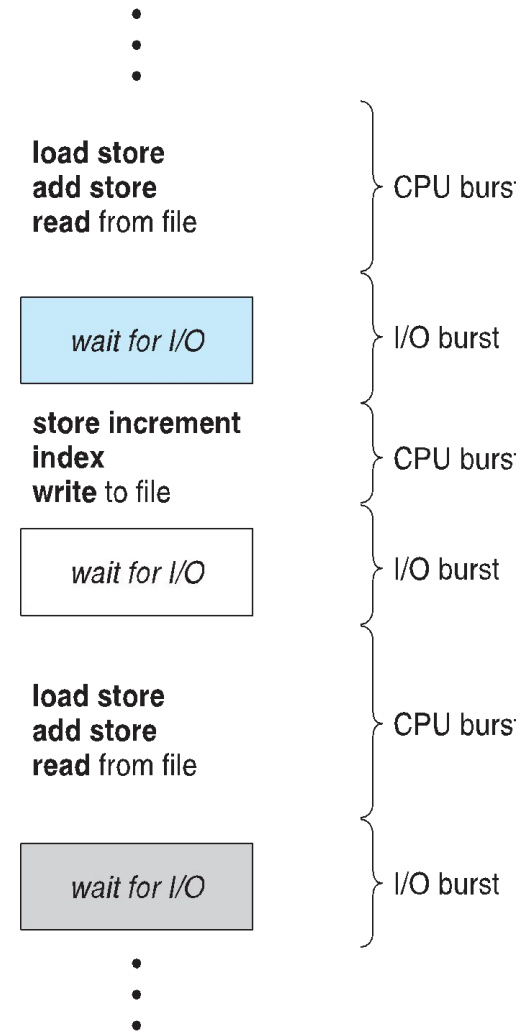


CPU Scheduling

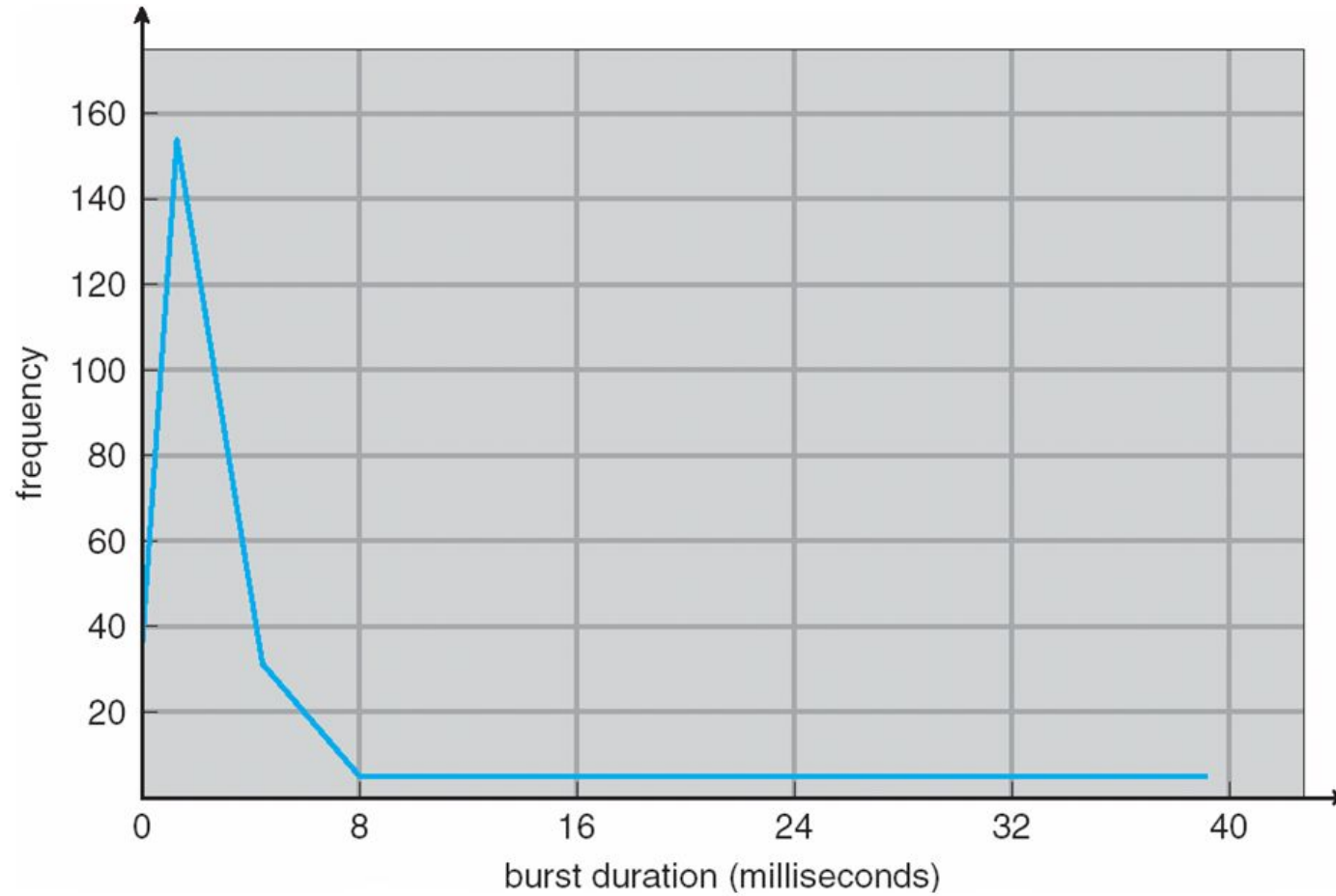
- **CPU Scheduling:** Scheduling algorithms, context switching, impact on AI/ML workloads (Week 3)

Basic Concepts

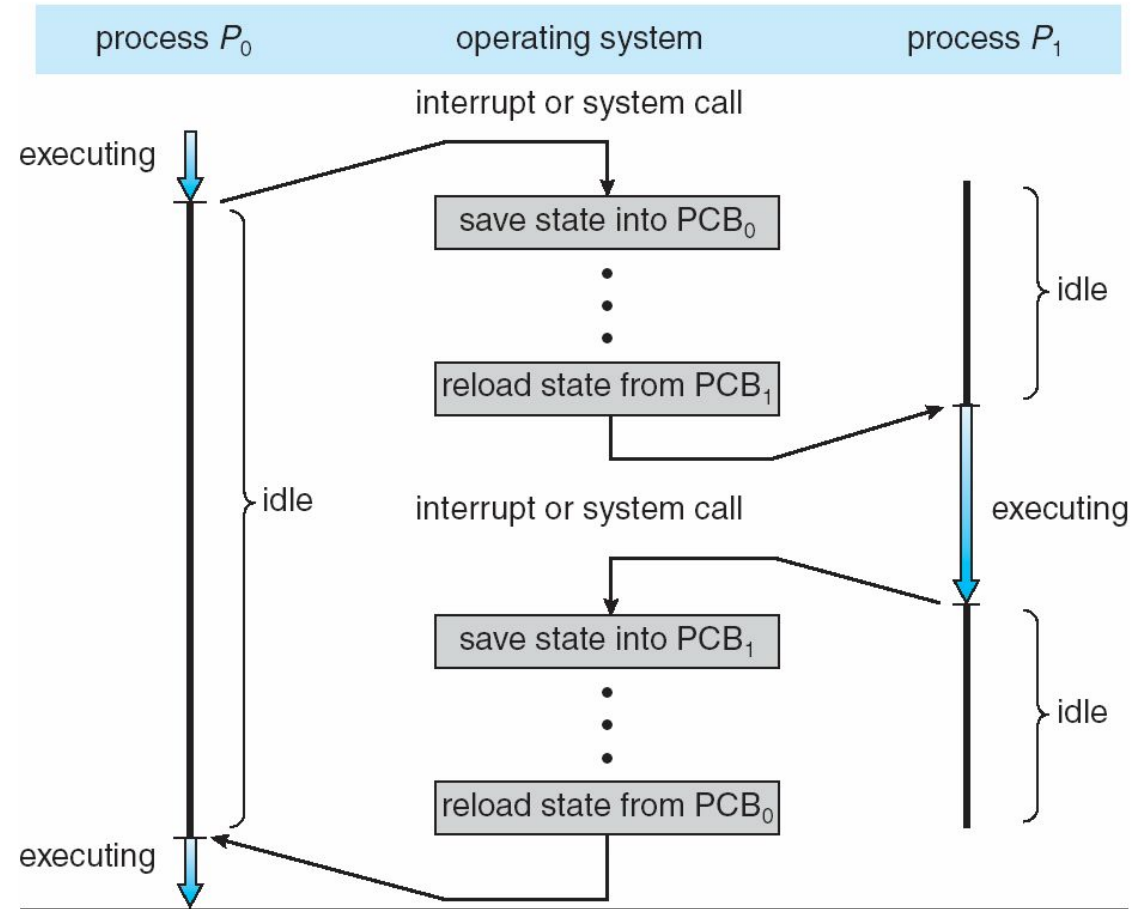
- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- CPU burst distribution is of main concern



Histogram of CPU-burst Times



CPU Switch From Process to Process



Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
(Turnaround Time = Completion Time – Arrival Time)
- **Waiting time** – the amount of time a process has been waiting in the ready queue
- **Response time** – the amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

First- Come, First-Served (FCFS) Scheduling

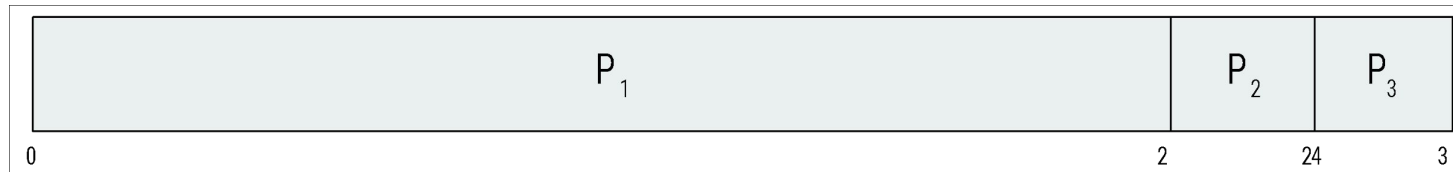
<u>Process</u>	<u>Burst Time</u>
----------------	-------------------

P_1	24
-------	----

P_2	3
-------	---

P_3	3
-------	---

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

P_2, P_3, P_1

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6; P_2 = 0; P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case

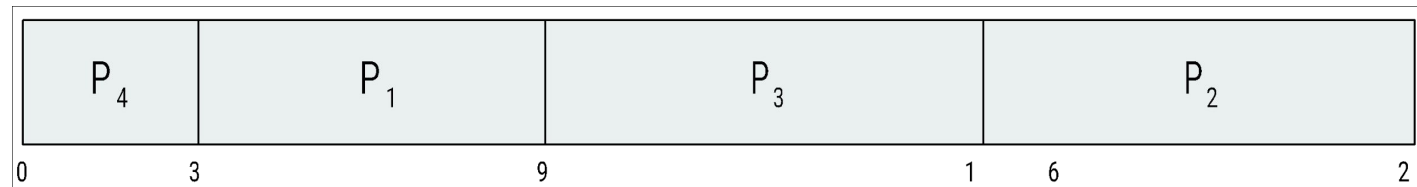
Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst
 - Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
 - The difficulty is knowing the length of the next CPU request
 - Could ask the user

Example of SJF

	<u>Process</u>	<u>Burst Time</u>
P_1	6	
P_2	8	
P_3	7	
P_4	3	

- SJF scheduling chart



- Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$

Determining Length of Next CPU Burst

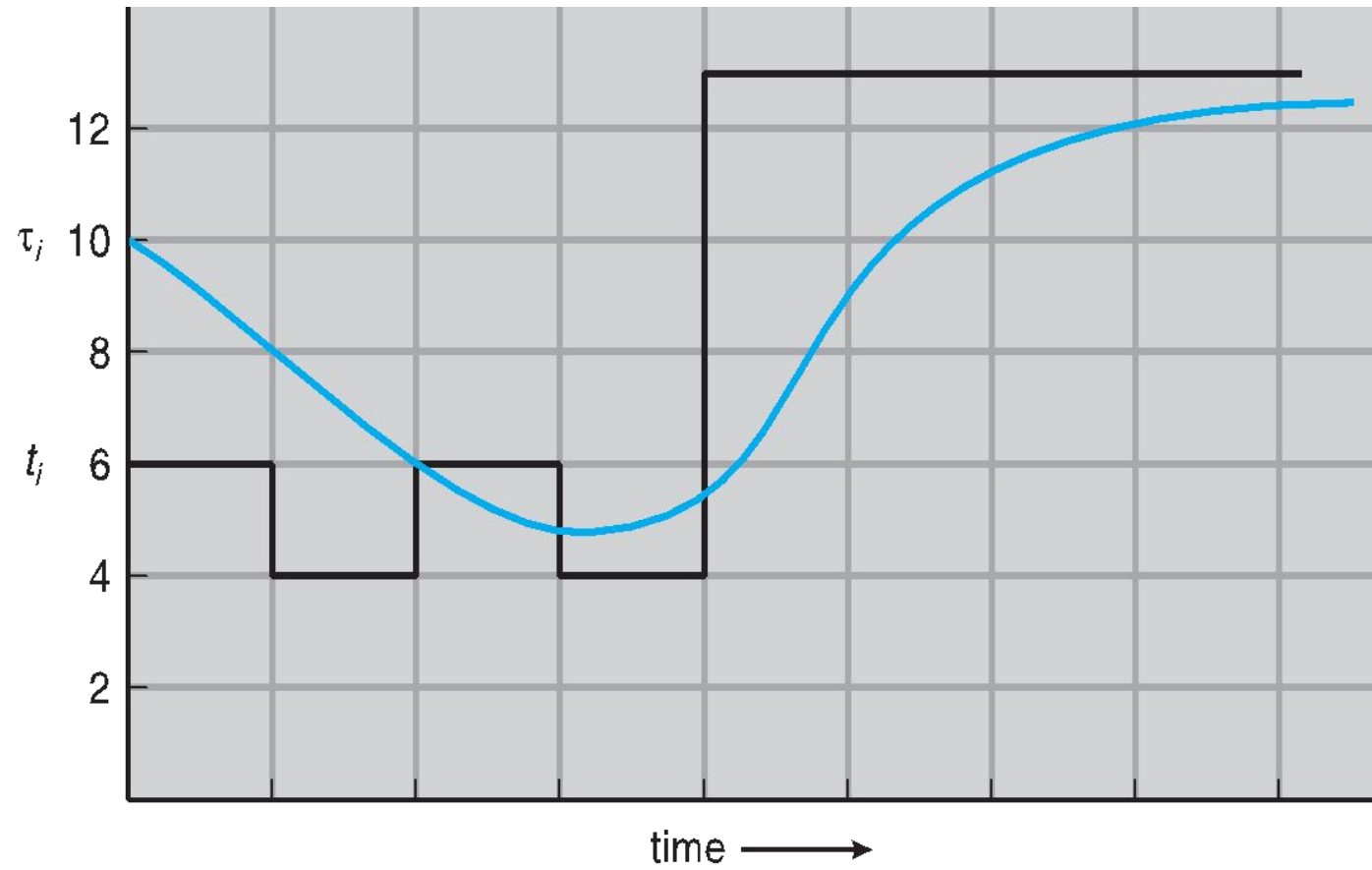
- Can only estimate the length – should be similar to the previous one
 - Then pick a process with the shortest predicted next CPU burst
- Can be done by using the length of previous CPU bursts, using exponential averaging

1. t_n = actual length of n^{th} CPU burst
2. τ_{n+1} = predicted value for the next CPU burst
3. $\alpha, 0 \leq \alpha \leq 1$
4. Define:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n.$$

- Commonly, α is set to $\frac{1}{2}$
- Preemptive version called **shortest-remaining-time-first**

Prediction of the Length of the Next CPU Burst



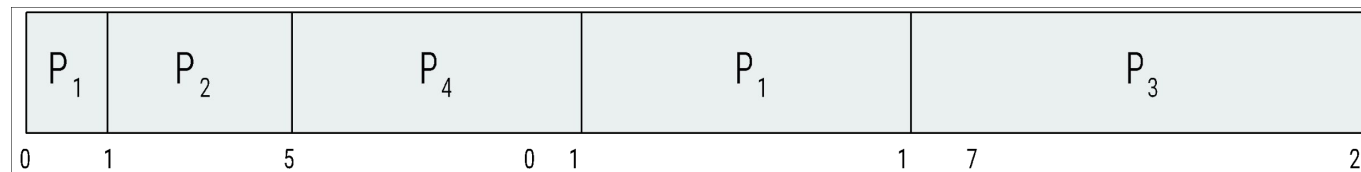
CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

Example of Shortest-remaining-time-first

- Now we add the concepts of varying arrival times and preemption to the analysis

	<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8	
P_2	1	4	
P_3	2	9	
P_4	3	5	

- Preemptive* SJF Gantt Chart



- Average waiting time = $[(10-1)+(1-1)+(17-2)+5-3)]/4 = 26/4 = 6.5$ msec

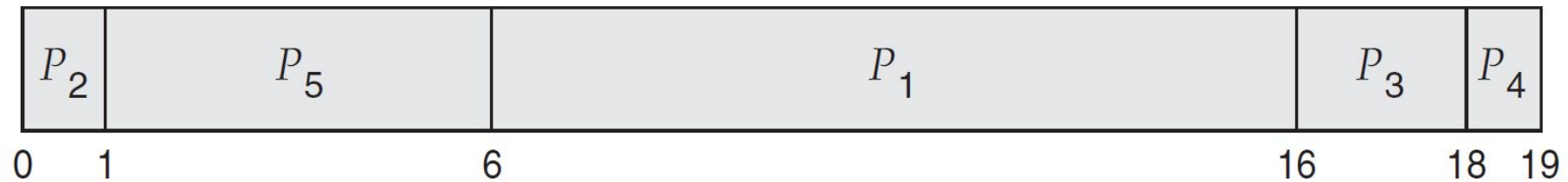
Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
 - Preemptive
 - Nonpreemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem $\equiv$ **Starvation** – low priority processes may never execute
- Solution $\equiv$ **Aging** – as time progresses increase the priority of the process

Example of Priority Scheduling

	<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3	
P_2	1	1	
P_3	2	4	
P_4	1	5	
P_5	5	2	

- Priority scheduling Gantt Chart



- Average waiting time = 8.2 msec

Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum q**), usually 10-100 milliseconds.
- After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule next process
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

Example of RR with Time Quantum = 4

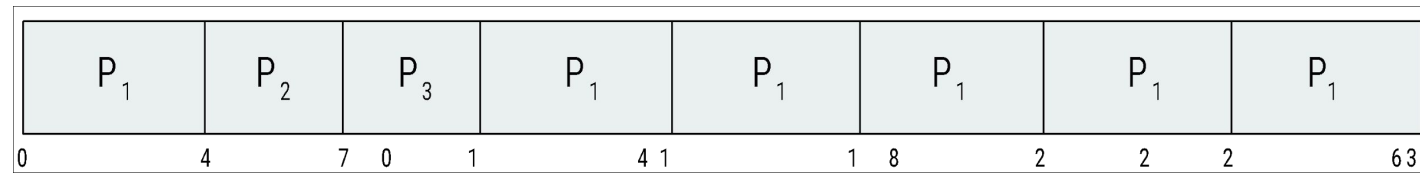
Process Burst Time

P_1 24

P_2 3

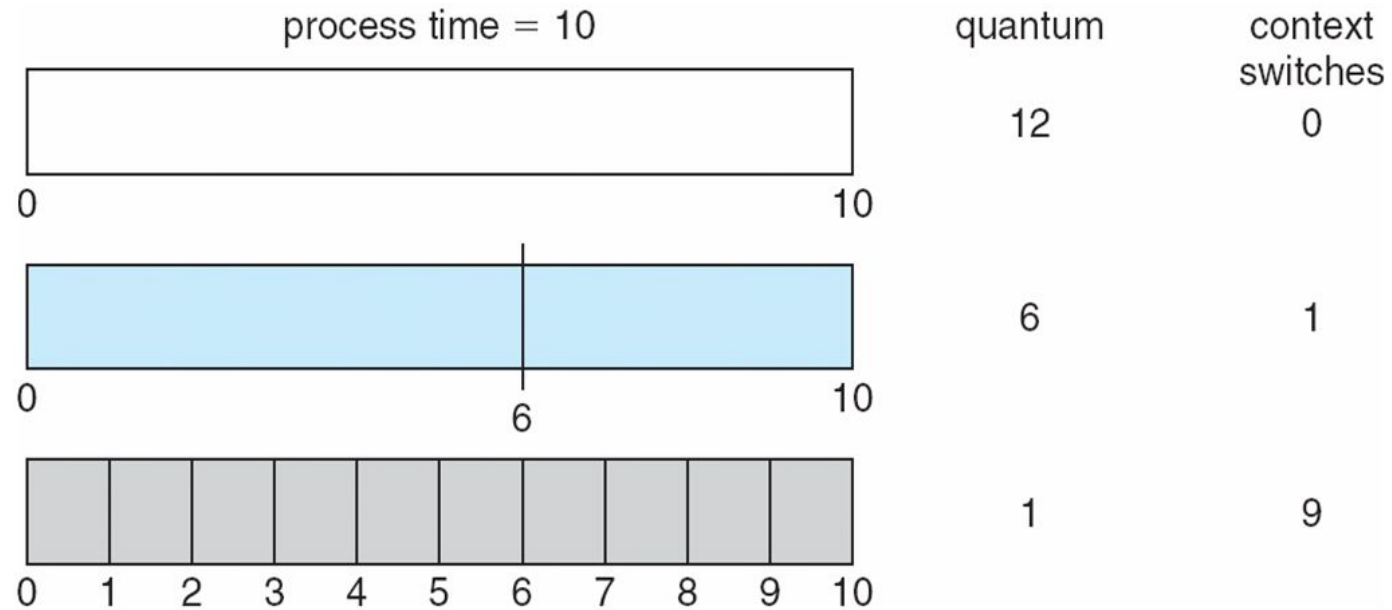
P_3 3

- The Gantt chart is:

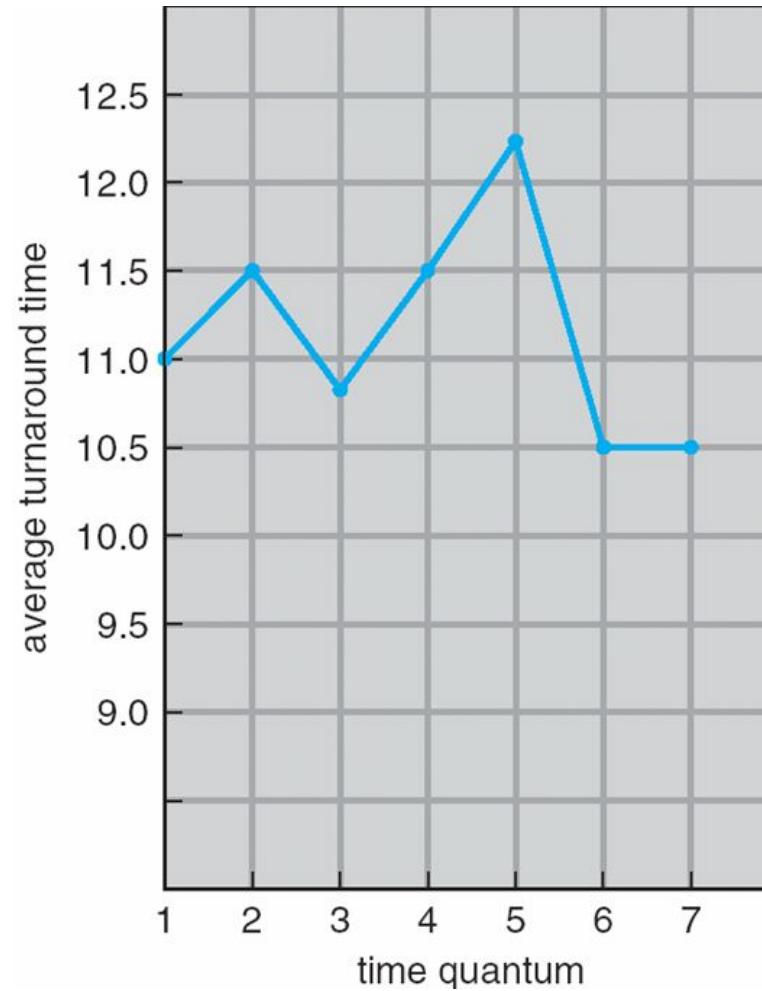


- Typically, higher average turnaround than SJF, but better **response**
- q should be large compared to context switch time
- q usually 10ms to 100ms, context switch < 10 usec

Time Quantum and Context Switch Time



Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

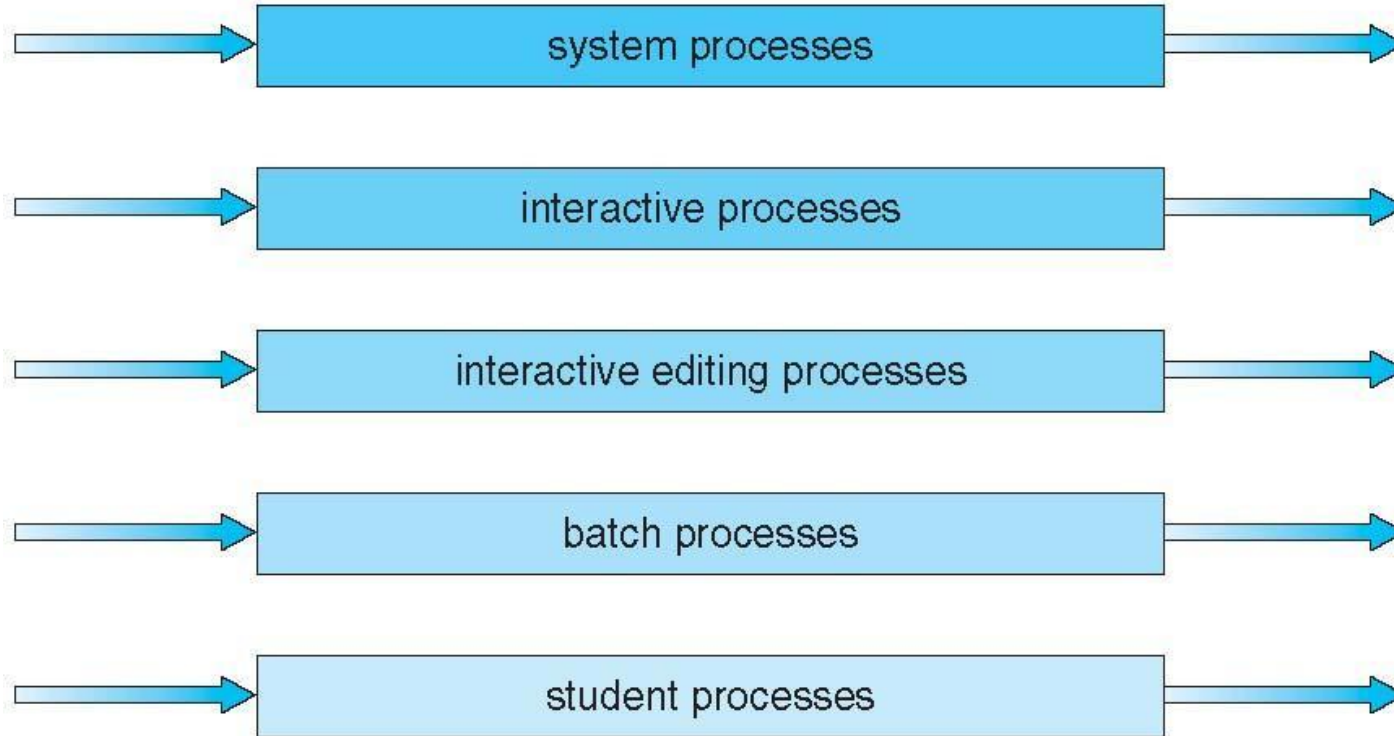
80% of CPU bursts
should be shorter than q

Multilevel Queue

- Ready queue is partitioned into separate queues, eg:
 - **foreground** (interactive)
 - **background** (batch)
- Process permanently in a given queue
- Each queue has its own scheduling algorithm:
 - foreground – RR
 - background – FCFS
- Scheduling must be done between the queues:
 - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR; 20% to background in FCFS

Multilevel Queue Scheduling

highest priority



lowest priority

Multilevel Feedback Queue

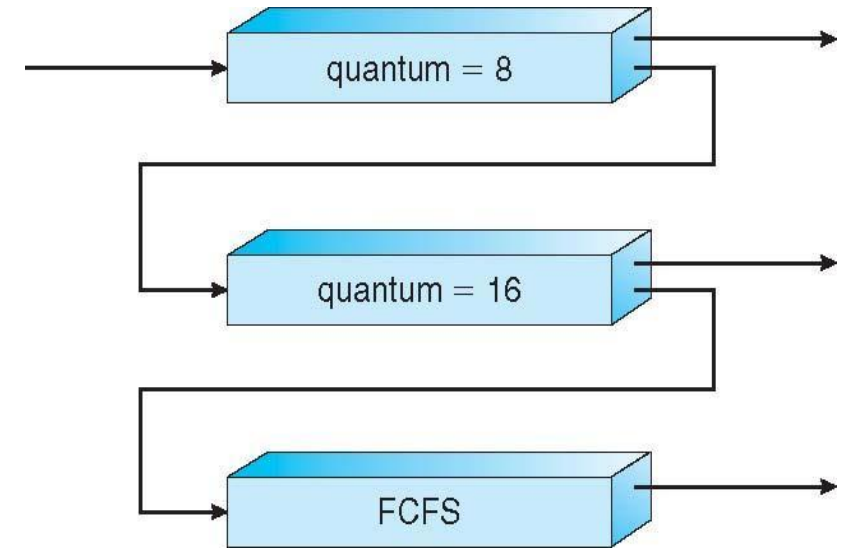
- A process can move between the various queues
- New process arrives, assign it to the top priority queue.
- If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down one queue).
- If a process yields or waits for I/O before the quantum expires, it stays in the same queue.
- Aging: Processes waiting too long in low queues may be promoted to prevent starvation.

Interactive jobs (short bursts) → finish quickly in higher queues.

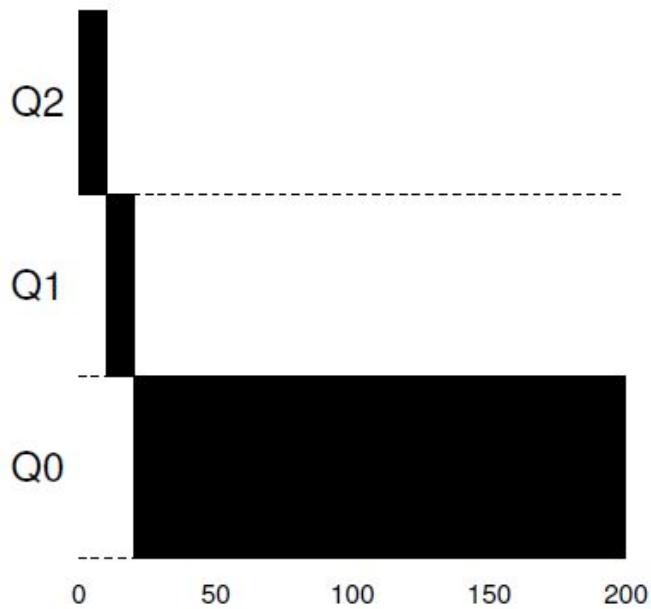
CPU-bound jobs → pushed down but still run eventually.

Example of Multilevel Feedback Queue

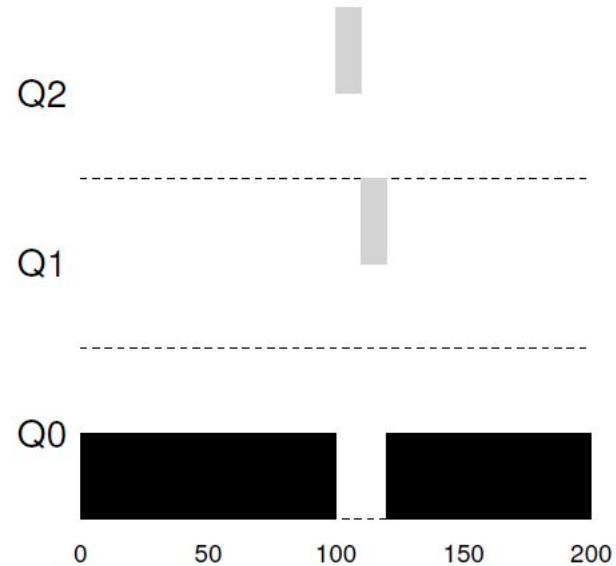
- Three queues:
 - Q_0 – RR with time quantum 8 milliseconds
 - Q_1 – RR time quantum 16 milliseconds
 - Q_2 – FCFS
- Scheduling
 - A new job enters queue Q_0 which is served RR
 - When it gains CPU, job receives 8 milliseconds
 - If it does not finish in 8 milliseconds, job is moved to queue Q_1
 - At Q_1 job is again served RR and receives 16 additional milliseconds
 - If it still does not complete, it is preempted and moved to queue Q_2



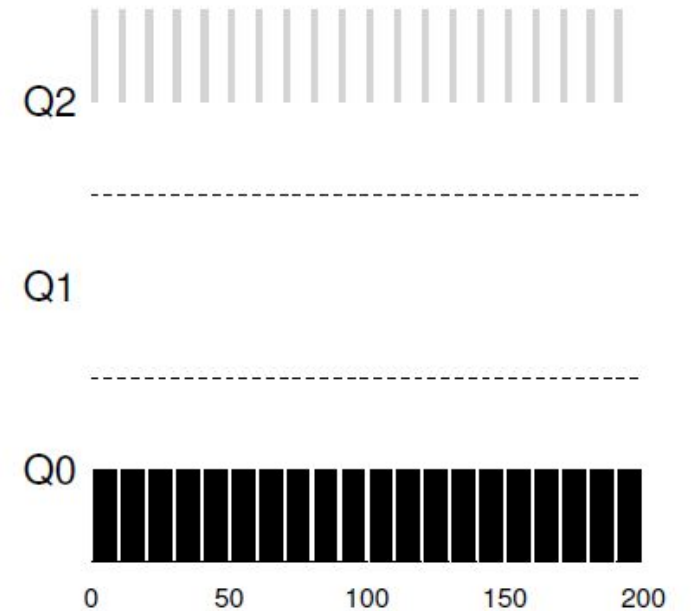
MLFQ: Scenarios



**Long-running Job
Over Time**



**Along Came
an Interactive
Job**



**A Mixed I/O-intensive
and
CPU-intensive
Workload**

MLFQ: Philosophy

- MLFQ doesn't know whether a job will be a short job or a long-running job; it first assumes it might be a short job. Thus giving the job high priority.
- If it actually is a short job, it will run quickly and be completed
- If it is not a short job, it will slowly move down the queues and thus soon prove itself to be a long-running, more batch-like process

MLFQ: Major issues and solutions

- **Problem:** if there are “too many” interactive jobs in the system, they will combine to consume all CPU time, and thus long-running jobs will never receive any CPU time
- **Solution:** After some time period S , move all the jobs in the system to the topmost queue.
- -----
- **Problem:** A smart user could rewrite their program to game the scheduler.
- **Solution:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).

Impact of scheduling on AI/ML workload

(a) Mixed compute and I/O phases

- AI workloads alternate between:
- **CPU-heavy phases:** data decoding, augmentation, tokenization
- **GPU-heavy phases:** forward/backward passes
- **I/O-heavy phases:** disk, network, checkpointing
- Poor scheduling causes phase mismatch, where CPU delays stall GPU pipelines.

(b) Bursty and synchronized execution

- DNN training uses **barrier-based collectives** (all-reduce, all-gather)
- All workers must reach the barrier together
- If one process is descheduled, **all GPUs wait** (straggler problem).

(c) Latency-sensitive inference paths

- First-token latency
- Autoregressive decode loops
- RPC handling threads
- Preemption or time-slice expiry increases tail latency.